

《编译原理》实验报告

姓名：李经雄

学院：计算机科学与技术学院

专业：计算机科学与技术

邮箱：3220105346@zju.edu.cn

报告日期：2024. 4. 13

Lab4 riscv 目标代码生成

一 实验内容

1.1 riscv 指令集定义

在asm.hpp中定义所有需要用到的riscv指令，包括乘法除法和取余等指令。特别地，我还新增了li加载立即数指令便于处理当立即数的大小限制，偏移量的大小限制等问题，比如lw/sw指令遇到过大偏移量的时候就会利用li指令把偏移量加载到寄存器中再使用。

1.2 指令选择

指令的选择需要对每一条中间代码进行指令的翻译，我新增了下面这些翻译函数，许多指令的翻译其实是显然的。

```
1  ASM::Code selectLoad(const IR::LoadPtr &node);
2  ASM::Code selectStore(const IR::StorePtr &node);
3  ASM::Code selectIf(const IR::IfPtr &node);
4  ASM::Code selectParam(const IR::ParamPtr &node);
5  ASM::Code selectDec(const IR::DecPtr &node);
6  ASM::Code selectAddressOf(const IR::AddressOfPtr &node);
```

在对一些含有立即数或者偏移量的指令的时候需要进行特殊处理，翻译为li指令将超大立即数加载进寄存器之后再用寄存器做加法。实现汇编代码翻译之后还需要实现寄存器分配、栈帧布局等任务。

```
1  ASM::Code InstSelector::selectLoadImm(const IR::LoadImmPtr &node) {
2      ASM::Code code;
3      // 需要判断立即数大小是否超过 12 位 超过的话要特殊处理
4      if (node->k > 2047 || node->k < -2048) {
5          code.push_back(ASM::Lim::create(ASM::Reg::t0, node->k));
6          code.push_back(ASM::Arith::create(ASM::Reg::t0, ASM::Reg::zero, ASM::Reg::t0,
7      ASM::Arith::Op::Add));
8      } else {
9          code.push_back(ASM::ArithImm::create(ASM::Reg::t0, ASM::Reg::zero, node->k,
10     ASM::ArithImm::Op::Addi));
11     }
12     int offset_rd = get_temp_offset(node->x);
13     code.push_back(ASM::Store::create(ASM::Reg::sp, ASM::Reg::t0, offset_rd));
14     return code;
15 }
```

二 实现亮点

2.1 栈帧布局

在lab3中，代码框架帮助实现了一个最基本的基本块划分，每一个函数基本块作为分配一块栈帧的基本单位。这里实现了类似实验指导中所描述的那种栈帧布局，在翻译汇编代码的时候调用alloc_temp分配相应的栈帧空间，返回的是相对sp的偏移量。因为在lab4中我实现的是最基本的寄存器分配算法，即仅使用到三个寄存器而已，每当遇到一个新的临时变量就在栈上为他分配空间，使用它的时候再从栈上load对应的值，所以这里使用了一个temp_var_offset_map将临时变量映射为相对sp的偏移量，这样就可以比较简单地实现翻译汇编代码。

```

1  int Function::alloc_temp(int size) {
2      // 为临时变量分配空间，临时变量通常分配在靠近 sp 的位置
3      // 返回值是相对于 sp 的偏移
4      int temp_stack_size_old = temp_stack_size;
5      temp_stack_size += size;
6      return temp_stack_size_old;
7  }
8  int InstSelector::get_temp_offset(const std::string &name) {
9      auto it = current_func->temp_var_offset_map.find(name);
10     if (it == current_func->temp_var_offset_map.end()) {
11         int offset = current_func->alloc_temp();
12         current_func->temp_var_offset_map[name] = offset;
13         return offset;
14     } else {
15         return it->second;
16     }
17 }

```

2.2 多函数参数

本次实验还需要处理函数参数大于8个的情况，我在这里也是采取了比较简单粗暴的解决方法，即直接在栈帧底部留出一些额外的空间供传参。InstSelector类中提供了一个当前函数基本块的指针，我新增了两个变量，一个代表实参个数一个代表形参个数。如果参数不超过8个，则直接使用a0-a7寄存器传参。当实参个数超过8个后，在栈帧低地址处预留的30字节空间依次存储对应的实参，注意遇到Call指令之后应该把arg_count重置为0，当形参个数超过8个后，需要从调用者函数的栈帧中获取存放在栈上的变量，所以使用fp进行load操作，注意fp和上一个函数sp的关系式fp=sp_pre - 4所以存取偏移量有一个字长的差别。

```

1  FunctionPtr current_func;
2  int arg_count = 0;
3  int param_count = 0;
4  ASM::Code InstSelector::selectArg(const IR::ArgPtr &node) {
5      ASM::Code code;
6      int offset_rs = get_temp_offset(node->x);
7      code.push_back(ASM::Load::create(ASM::Reg::t1, ASM::Reg::sp, offset_rs));
8      if (arg_count < 8) {
9          std::string reg_name = "a" + std::to_string(arg_count);
10         code.push_back(ASM::Mv::create(ASM::Reg(reg_name), ASM::Reg::t1));
11     } else {
12         // 如果参数超过了 8 个，使用栈来传递参数
13         // 栈上面已经预留了从 sp 开始的 30 个字节的空间 依次存储即可
14         code.push_back(ASM::Store::create(ASM::Reg::sp, ASM::Reg::t1, 4 * (arg_count -
15 8)));
16     }
17     arg_count++;
18     return code;
19 }
20 ASM::Code InstSelector::selectParam(const IR::ParamPtr &node) {
21     ASM::Code code;
22     if (param_count < 8) {
23         std::string reg_name = "a" + std::to_string(param_count);
24         code.push_back(ASM::Mv::create(ASM::Reg::t0, ASM::Reg(reg_name)));
25     }
26     param_count++;
27     return code;
28 }

```

```

24     } else {
25         code.push_back(ASM::Load::create(ASM::Reg::t0, ASM::Reg::fp, 4 * (param_count -
26         7)));
27     }
28     int offset = get_temp_offset(node->x);
29     code.push_back(ASM::Store::create(ASM::Reg::sp, ASM::Reg::t0, offset));
30     param_count++;
31     return code;
32 }

```

2.3 汇编生成

最终生成汇编还需要一个AsmEmitter类，这个类主要是处理好整个翻译好的汇编并生成可以运行的版本。这里重要的地方是生成全局变量和处理函数的prologue和epilogue的生成。添加好.data之后根据中间代码生成好的全局变量初始值，为每一个全局变量的word赋值。进入函数和退出函数时，需要操作ra, sp, fp等寄存器，这里也需要考虑偏移量是否超出限制的问题，所以源码中还需要更进一步进行判断。

```

1  void ASMEmitter::emit(const Module &mod) {
2      // 添加 .data 段
3      output << ".data" << std::endl;
4      for (const auto &global : mod.global_ir) {
5          if (auto global_var = std::dynamic_pointer_cast<IR::Global>(global)) {
6              output << global_var->name << ":" << std::endl;
7              for (const auto &val : global_var->init_vals) {
8                  output << "    .word " << val << std::endl;
9              }
10         }
11     }
12     output << "    .section .text" << std::endl;
13     for (const auto &func : mod.functions) {
14         emit(func);
15     }
16 }
17 void ASMEmitter::emit(const FunctionPtr &func) {
18     int stack_size = func->temp_stack_size + func->reg_stack_size;
19     reg_map = func->reg_map; // 设置当前函数的寄存器映射
20     // 添加 prologue, 处理 sp, ra, fp 等寄存器
21     output << ".globl " << func->name << std::endl;
22     output << func->name << ":" << std::endl;
23     output << "    addi sp, sp, -" << stack_size << std::endl;
24     output << "    sw ra, " << (stack_size - 4) << "(sp)" << std::endl;
25     output << "    sw fp, " << (stack_size - 8) << "(sp)" << std::endl;
26     output << "    addi fp, sp, " << (stack_size - 4) << std::endl;
27     for (size_t i = 0; i < func->blocks.size() - 1; i++) {
28         emit(func->blocks[i]);
29     }
30     output << "    lw ra, " << (stack_size - 4) << "(sp)" << std::endl;
31     output << "    lw fp, " << (stack_size - 8) << "(sp)" << std::endl;
32     output << "    addi sp, sp, " << stack_size << std::endl;
33     output << "    ret" << std::endl;
34 }

```

三 AI工具使用

- ChatGPT：将框架代码交给ChatGPT分析，并让他解释代码框架的思路和我所需要完成的任务。描述待完成工作并让ChatGPT生成可以采取的实现形式供参考
- Copilot：辅助完成代码实现和注释

四 建议

完结咯!!! 接下来继续努力继续挑战bonus，实现一下更好的寄存器分配算法。